

Sprint 4 – Animation

Student Information

Integrity Policy: All university integrity and class syllabus policies have been followed. I have neither given, nor received, nor have I tolerated others' use of unauthorized aid.

I understand and followed these policies: Yes No

Name:

Date:

Submission Details

Final **Changelist** number:

Verified build: Yes No

Required Configurations:

YouTubeLink:

Discussion (What did you learn):

Verify Builds

- Follow the Piazza procedure on submission
 - Verify your submission compiles and works at the changelist number.
- Verify that only MINIMUM files are submitted
 - No – Generated files
 - *.pdb, *.suo, *.sdf, *.user, *.obj, *.exe, *.log, *.pdb, *.db, *.user
 - Anything that is generated by the compiler should not be included
 - No – Generated directories
 - /Debug, /Release, /Log, /ipch, /.vs
- Typical files project files that are required
 - *.sln, *.csproj, *.cs,
 - App.config, AssemblyInfo.cs, CleanMe.bat
 - Resources Directory:
 - *.tga, *.dll, *.wav, *.gls, *.azul

Standard Rules

Submit multiple times to Perforce

- Submit your work as you go to perforce several times
 - Design Patterns – no minimum
 - Sprints – minimum of 5 real submissions (generally 10+)
 - As soon as you get something working, submit to perforce
 - Have reasonable check-in comments
 - Points will be deducted if minimum is not reached

Submission Report

- Fill out the submission Report
 - No report, no grade

Code and project needs to compile and run

- Make sure that your program compiles and runs
 - Warning level 4
 - NO Warnings or ERRORS
 - Your code should be squeaky clean.
 - Code needs to work “as-is”.
 - No modifications to files or deleting files necessary to compile or run.
 - All your code must compile from perforce with no modifications.
 - Otherwise it's a 0, no exceptions

Project needs to run to completion

- If it crashes for any reason...
 - It will not be graded and you get a 0

No Containers

- Containers (No automatic containers or arrays)
- Template or generic parameters
- No arrays
 - You need to do this the old fashion way - **YOU EARNED IT**

Leave Project Settings

- Do NOT change the project or warning level
 - Any changing of level or suppression of warnings is an integrity issue

Simple C#

- No .Net
- We are using the basics
 - Types:
 - Class, Structs, intrinsic types (int, float, bool, etc...)
 - NO arrays allowed!
 - Basics language features
 - Inheritance, methods, abstract, virtual, etc...

No Debug code or files disabled

- Make sure the program has only active code
 - If you added debug code or commented out code,
 - please return to code to active state or remove it

Adding files to this project

- Make sure you add the files in the appropriate sub-directories
- Make sure any new files are successfully integrated into the project
- Make sure your new files are submitted to Perforce

Due Dates

- See Piazza for due date and time
- Submit program perforce in your student directory assignment supplied.
- Fill out your this **Submission Report** and commit to perforce
 - **ONLY** use Adobe Reader to fill out form, all others will be rejected.
 - Fill out the form and discussion for full credit.

Goals

- Refactor
 - Create **SpriteGameProxy** to use in the game
 - Create Timer system used in the Game
 - All the Timer Events need to be inserted in Triggered time order
 - Using **Command pattern** and **Priority Queue**
 - Create **Sprite Animation** used in the Game

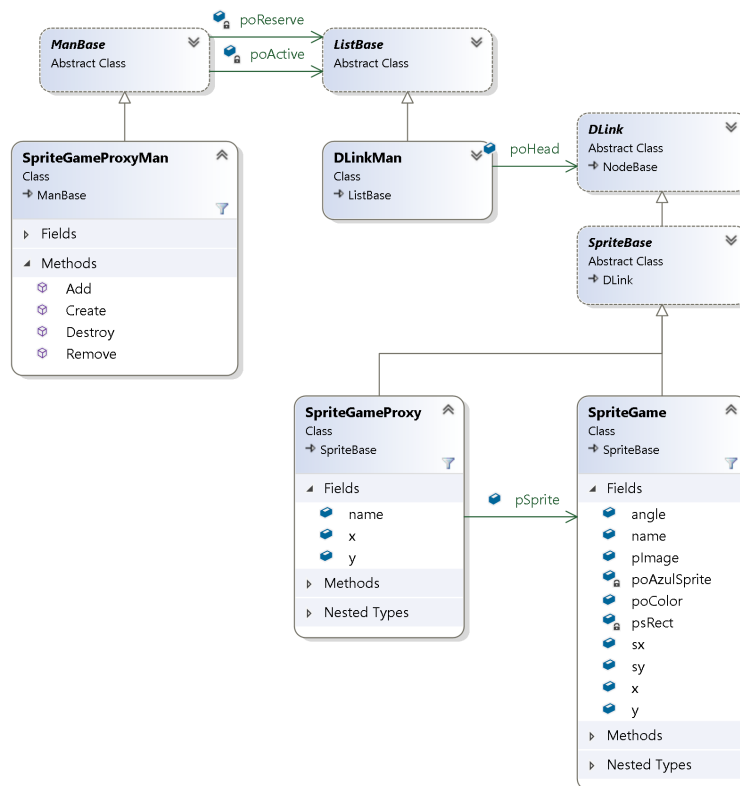
Assignments

Grading

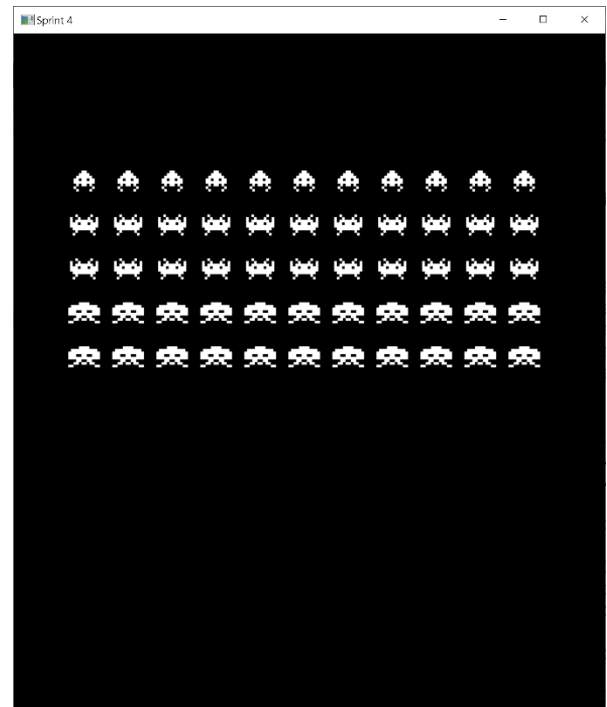
- This sprint you are going to learn/modify the code from class.
- There are 3 steps to this refactoring

Step 1 – Add Proxy Sprites

- Create a **SpriteGameProxy Manager**
 - **SpriteGameProxy** is a light weight sprite
 - Only unique data in the **Proxy** is its name and position (x,y)
 - This manager uses the **Proxy pattern**
 - Create a **Factory** to setup the 55 Aliens (rows x columns): 5 x 11

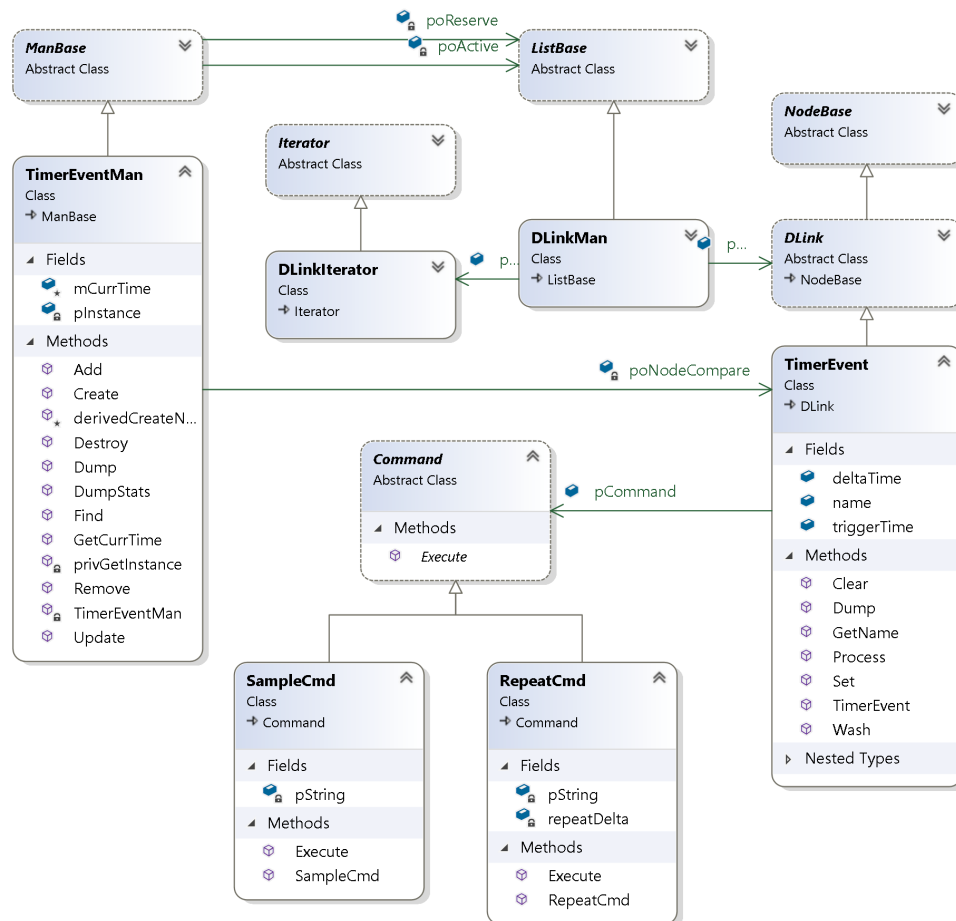


- Rendering
 - Screen Size and Color
 - 672 x 768
 - Original game was 224 x 256
 - We are creating a window that 3x the size
 - Everyone's monitor / laptop can hand this resolution
 - Clear Color
 - Black (RGB) (0,0,0) Alpha(1)
 - Create a 11x5 Grid of aliens
 - Use the new **SpacInvaders_ROM.tga**
 - **SpacInvaders_ROM.txt** – has the coordinates of the images
 - Image dimensions are in this text file
 - Remember (in-case you forget)
 - Top row – squid
 - Middle 2 rows – crab
 - Bottom 2 rows – octopus
 - Layout of Game Objects
 - GameObject (width x height)
 - Squid size: 24x 25
 - Crab size: 32 x 25
 - Octopus size: 36 x 25
 - Game object are arranged
 - 50 pixels apart horizontally
 - 50 pixels apart vertically
 - Grid of 5 rows x 11 columns
 - Bottom left Alien (Octopus)
 - start at (80, 400)



Step 2 – Timer System

- Create a Timer System
 - Create a Singleton **Timer Event manager**
 - With object pooling
 - **TimerEvent** class – uses the **Command pattern**
 - Trigger time (time to executed the event)
 - Delta time (time to add itself back to the time)
 - All TimerEvents use Command pattern to execute the event
 - Execute() command - for the use of the command pattern
 - Triggers the event when the trigger time has elapsed
 - Adding TimerEvents
 - Placed in **Sorted** order on the manager
 - Do not call sort methods...
 - Install the Events in the correct position base on Trigger time
 - Timer Event manager is a **Priority Queue** pattern
 - All Timer events are in the correct order
 - **Use the global time to trigger the sprite animations**
 - **EARLY** out when the iterator hits a time greater than global time
 - **Must show this section of code in video**



Step 3 – Add animation

- **Sprite Animation:**
 - Sprite Animation Command that cycles the Aliens sprites between their two Animated Images
 - Associate Images to this AnimationSprite command
 - Do this for all 3 Alien Types (one of each)
 - Squid - at 0.25 sec interval
 - Crab - at 0.5 sec interval
 - Octopus - at 1.0 sec interval
 - Add 5 sample Timer Events
 - At trigger time 500, 510, 520, 530, 540 seconds
 - Execute():
 - Print on to debug window
 - (Debug.WriteLine("---LATE COMMAND --"))
 - These extra timer events are there
 1. to show case that you are indeed early exit
 2. that they are installed in the correct order.
 - Add a quick `TimerEventManager.Dump()` at the bottom of `LoadContent()`
 - Show the debug window in talk
 - Remember (in-case you forget)
 - Top row – squid
 - Middle 2 rows – crab
 - Bottom 2 rows – octopus
 - See Video for animation
 - <https://www.youtube.com/watch?v=UOMnnftQvr0>

Required Features

Features to show off in Demo

Show it off and talk about these required features

- Show off the 3 steps
 - Demo and talk in the video
 - Need to See:
 - Insert TimerEvents based on Trigger
 - EARLY exit in the TimerEventManager
 - Proxy UML you are using
 - Video of it running
 - Should be close to the reference demo
- Show it off
 - Be proud of what you did

Video Capture

- Have Visual Studio open and ready to go.
- Start recording
 - Show output window running
 - Speak clearly and loud
 - Discuss the features you added in this demo
 - Duration
 - Approx. 2-3 min long YouTube video capture
 - Do not exceed 5 min
 - Use Zoom recording
 - High resolution
 - Good sound quality
 - Show on the capture... it compiling and then running
 - Place YouTube link in PDF
 - Make sure it public and runs without passwords or login
 - Watch your video
 - Can you see the code clearly
 - Can you hear your voice clearly
- Don't lose points
 - Make sure everything is submitted properly
 - Don't record beyond 5 min.
 - Your audio volume was too quiet (test it and verify it)
 - Your YouTube link needs to work "As-IS" with no login
 - Don't make it private... make it unlisted
 - You check-in too many files or temporary files
 - Make sure you run CleanMe.bat before perform submission

General guidelines:

- Post on Piazza questions and clarifications
- Make sure you delete these using directives (we are not using them)
 - `using System.Collections.Generic;`
 - `using System.Linq;`
 - `using System.Text;`
 - `using System.Threading.Tasks;`

Development

- Copy your Sprint3 into Sprint4.
 - Submit to perform and start your work
- Do not copy temporary files or submit extra data
- Make sure you run CleanMe.bat before adding to perform

Submission

- Submit your SprintX directory into performce:
 - /student/<yourname>/SprintX/...
 - You need to submit a complete C# project
 - Solution, project and C# files (whatever it takes to build the project)
 - Do not submit anything that is auto generated
 - Run the supplied CleanMe.bat before submission
 - Should cleanup files
- Fill out the Submission report and submit that pdf to your student directory

Validation

Simple checklist to make sure that everything is submitted correctly

- Is the project compiling and running without any errors or warnings?
- Does the project runs **ALL** without crashing?
- Is the submission report filled in and submitted to performce?
- Follow the verification process for performce
 - Is all the code there and compiles “as-is”?
 - No extra files
- Submit the YouTube link to PDF

Hints

Most assignments will have hints in a section like this.

- Watch the lecture... look at the code supplied in class
- Study the projects in class lecture

Troubleshooting

- Print to the output window to help you debug
 - It really helps
- Ask for clarification on piazza